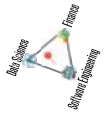
What does the "era" information mean for numer.ai modelling?



Era Boosting

```
train_eras = X
for r in boosting_round:
    train(train_eras)
    is_eval = evaulate(X)
    train_eras = worst_eras(is_eval)
```

Custom Objective

Era Splitting v1

200 Trees/s Custom CUDA Booster

warpGBM

1. written in only python and cuda
2. impements directional era splitting
3. does not need to copy the input data

What does it do?

The Era Splitting Criterion

$$\mathcal{L}_{split} = \frac{1}{2}\left[\frac{\left(\sum_{i\in I_L} g_i\right)^2}{\sum_{i\in I_L} h_i + \lambda} + \frac{\left(\sum_{i\in I_R} g_i\right)^2}{\sum_{i\in I_R} h_i + \lambda} - \frac{\left(\sum_{i\in I} g_i\right)^2}{\sum_{i\in I} h_i + \lambda}\right]$$

The Splitting Criterion

the sum of the gradients of the data

the sum of the 2nd order gradients of the data

The Splitting Criterion

Is it good?

The Era Splitting Criterion

The Era Splitting Criterion is the same as the original one, just done for each era seperately and then averaged together! The chosen split has the greatest average decrease of impurity over all eras.

The DIRECTION of a Split

A split divides the data into a smaller (left) and bigger (right) portion. The prediction value of any node is the mean of all contained samples.

So every split has a DIRECTION:
LEFT, if the prediction of the left child is bigger
RIGHT, if the prediction of the right child is bigger

One Dataset, no problem. But the direction can be different for each ERA.

Directional Era Splitting

```
def directional_agreement(split):
    aggregated_direction = 0
    for era in eras:
        aggregated_direction += direction(split, era)
    return aggregated_direction / len(eras)
```

All eras split in the same direction: `directional_agreement(split) = 1`
Half eras split in the same direction: `directional_agreement(split) = 0`

warpGBM Implementation

```
def choose_best_split(splits):
    max_agreement = max(directional_agreement(s) for s in splits)
    splits = [s for s in splits if directional_agreement(s) == max_agreement]
    return sorted(splits, key=split_criterion(), reverse=True)
```

ERA Bucket

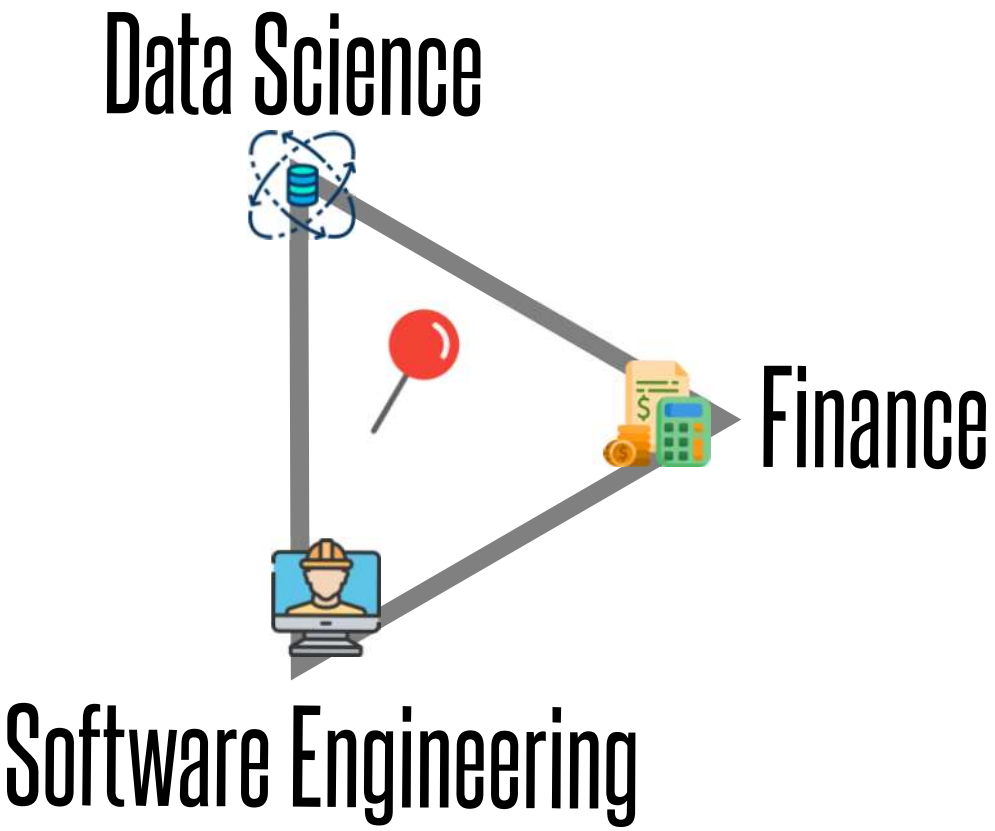
```
model = WarpGBM(...)
# no era splitting
era_id = np.ones(len(y))
# eras == buckets
era_id = numerai_data.eranо
assert len(era_id) == len(y)
assert era_id.all(istype(int))
model.fit(X,y,era_id)
```

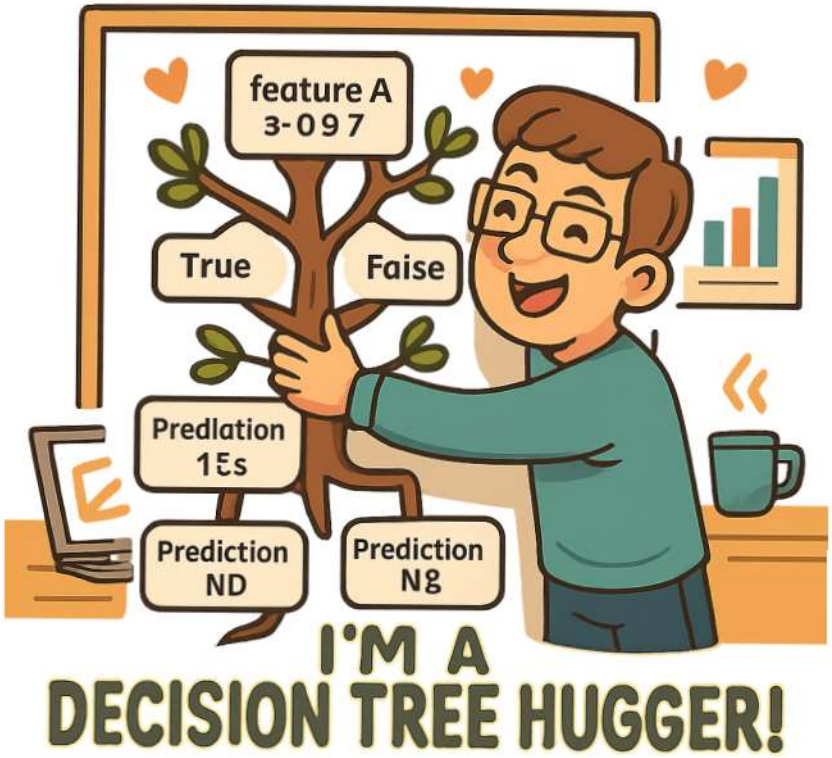
1 bucket for all data and 1 bucket for one era are the two extremes

GBRECHT

Joined 7 years ago

all4her


Data Science
Finance
Software Engineering


feature A
3-097
True
Faise
Predlation
1Es
Prediction
ND
Prediction
Ng
I'M A
DECISION TREE HUGGER!

Era Boosting

```
train_eras = X
for r in boosting_round:
    train(train_eras)
    is_eval = evaulate(X)
    train_eras = worst_eras(is_eval)
```

https://forum.numer.ai/t/era-boosted-models/189/1

http

Era Boosting

```
train_eras = X
for r in boosting_round:
    train(train_eras)
    is_eval = evaulate(X)
    train_eras = worst_eras(is_eval)
```

Boosting Algorithm selects highest LOSS gain splits

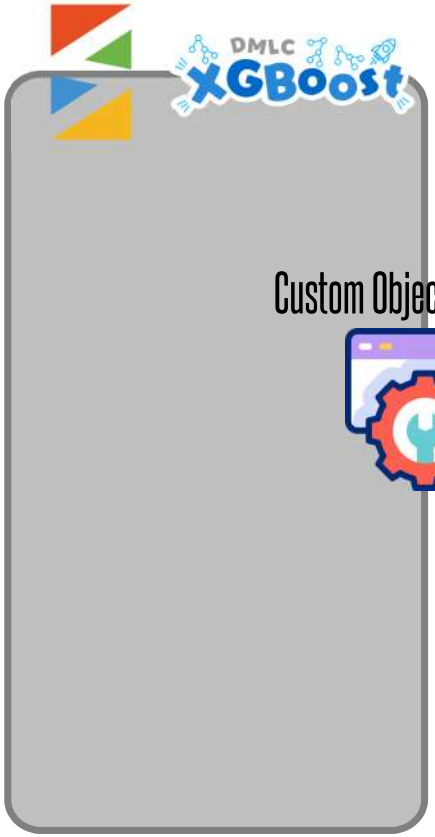
Next Iteration is trained on the error of the current ensemble

Evaluation on a METRIC (not LOSS)

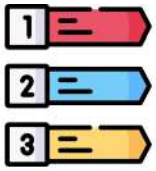
https://forum.numer.ai/t/era-boosted-models/189/1

http

Custom Objective



integer index of eras (not numerai-ids)



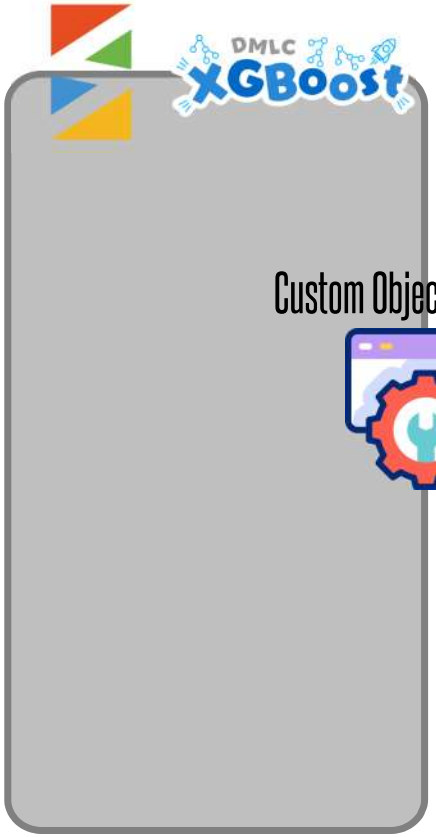
Custom Objective Function


```
def loss(preds, y)->gradient:
    global era_idx
    era_preds = preds[era_idx]
    era_truth = y[era_idx]
```

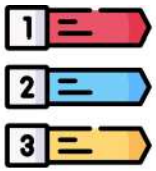
return 

https://forum.numer.ai/t/custom-loss-functions-for-xgboost-using-pytorch/960

Custom Objective



integer index of eras (not numerai-ids)



Custom Objective Function

```
def loss(preds, y)->gradient:
    global era_idx
    era_preds = preds[era_idx]
    era_truth = y[era_idx]
```

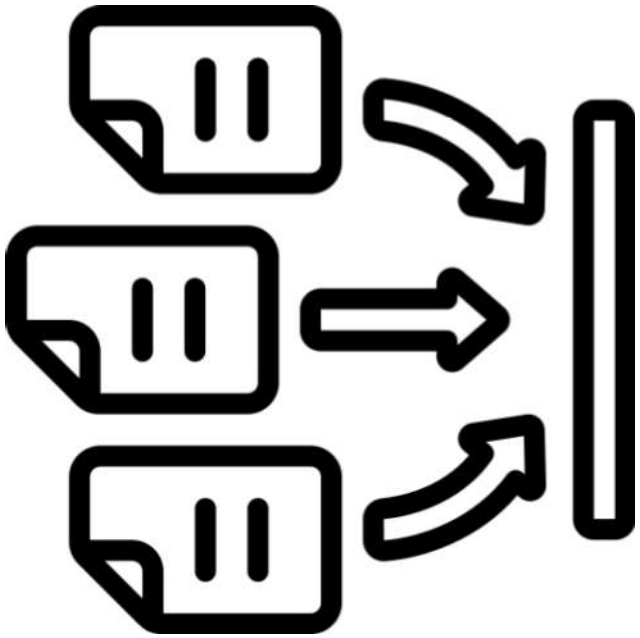
return 

https://forum.numer.ai/t/custom-loss-functions-for-xgboost-using-pytorch/960

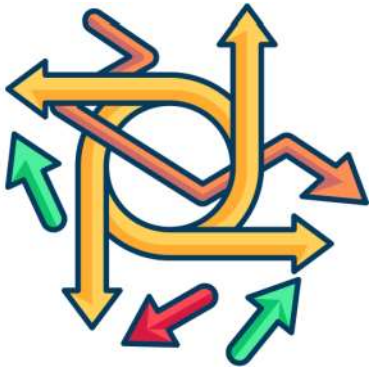

%
+
+
Σ
%
+
Σ
%
β
π
Σ
λ


GETTING A MODEL TO
LEARN A CUSTOM OBJECTIVE
LIVE PERFORMANCE
IS OPTIONAL

not separable means that the loss is not defined per sample, but per era.
so for 5 mio samples in 1000 eras, the algorithm does not get 5 mio values, but 1000

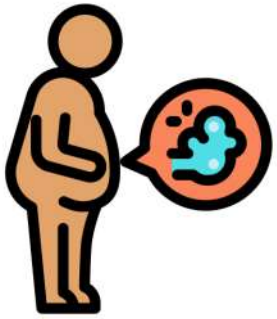


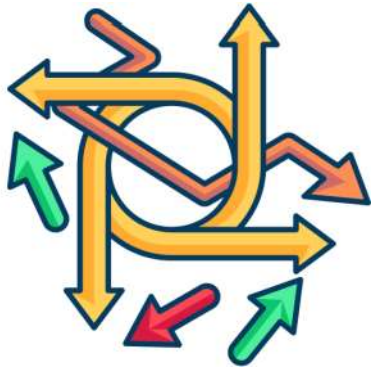
Era Splitting v1

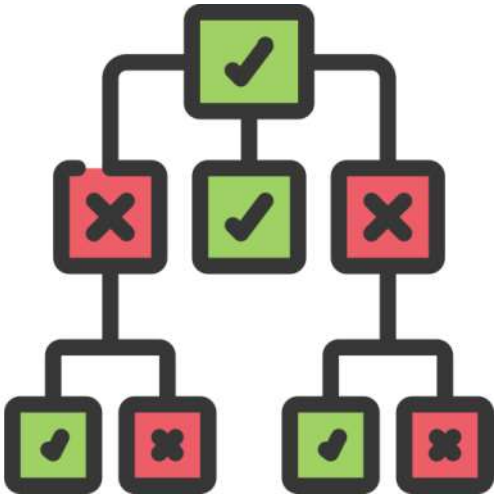


Paper: https://arxiv.org/abs/2309.14496
Code: https://github.com/jefferythewind/scikit-learn-erasplit
Quant Club: https://www.youtube.com/watch?v=H0mHxuRQy18
Forum: https://forum.numer.ai/t/era-splitting-invariant-learning-for-gradient-boosted-decision-trees/6690

Era Splitting v1







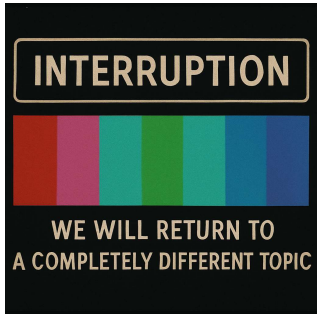
Paper: https://arxiv.org/abs/2309.14496
Code: https://github.com/jefferythewind/scikit-learn-erasplit
Quant Club: https://www.youtube.com/watch?v=H0mHxuRQy18
Forum: https://forum.numer.ai/t/era-splitting-invariant-learning-for-gradient-boosted-decision-trees/6690

200 Trees/s Custom CUDA Booster

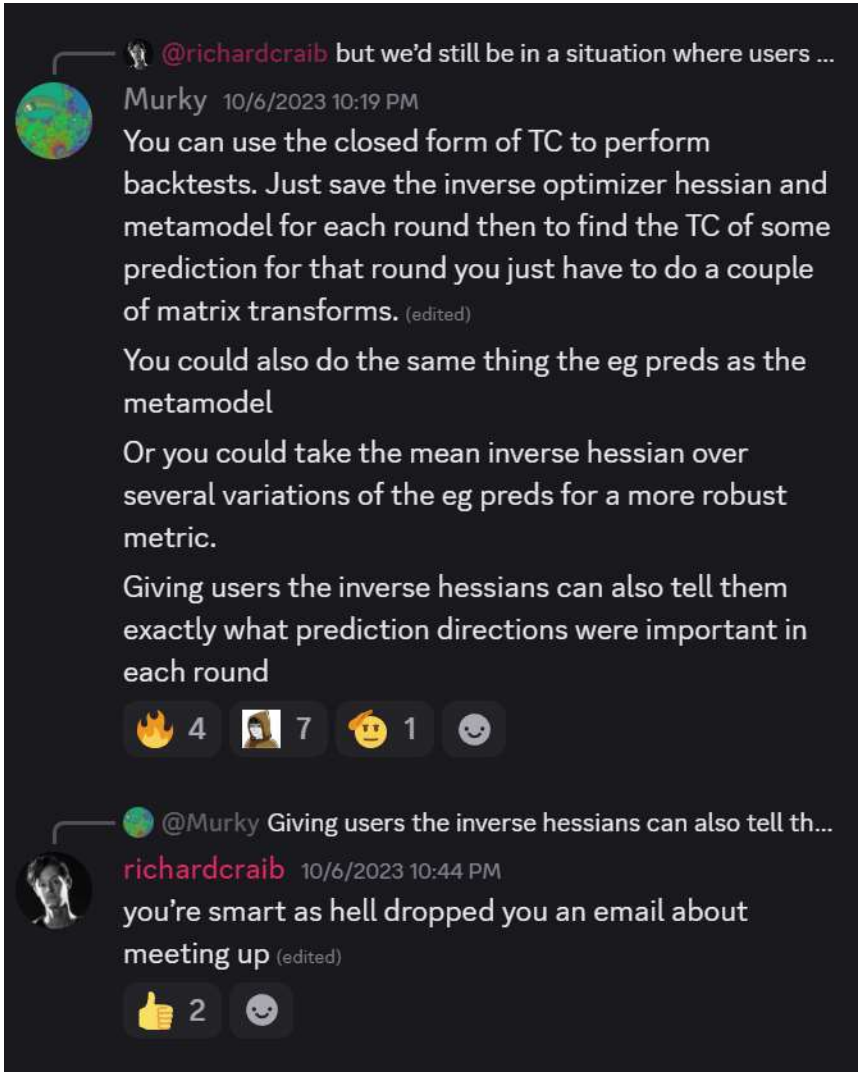
has nothing to do with eras

Forum 200 https://forum.numer.ai/t/200-trees-per-second-with-cuda/7823
200 Code https://colab.research.google.com/drive/1EU0a4mMLyUBCBLdjIS3_qEM85bHERtKi
Forum TC: https://forum.numer.ai/t/a-closed-form-expression-for-tc/6280

200 Trees/s Custom CUDA Booster

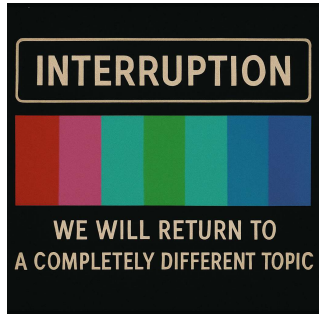


has nothing to do with eras



Forum 200 https://forum.numer.ai/t/200-trees-per-second-with-cuda/7823
200 Code https://colab.research.google.com/drive/1EU0a4mMLyUBCBLdjIS3_qEM85bHERtKi
Forum TC: https://forum.numer.ai/t/a-closed-form-expression-for-tc/6280

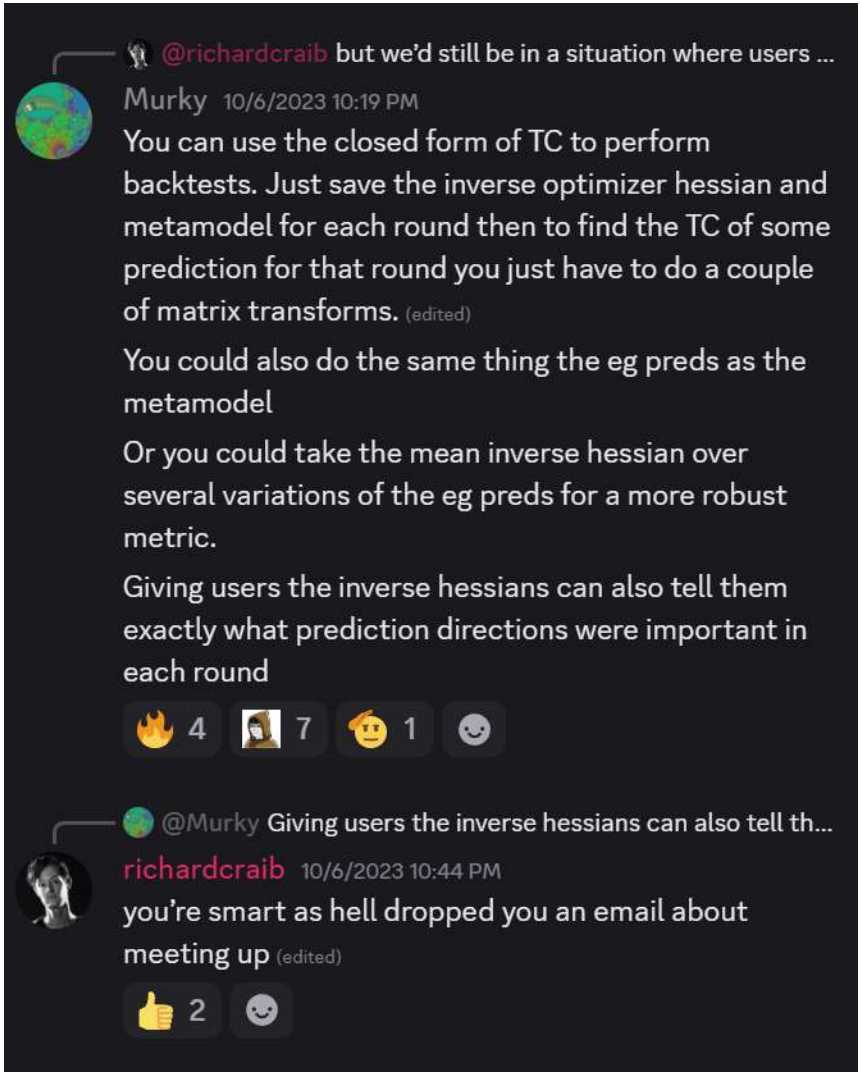
200 Trees/s Custom CUDA Booster



has nothing to do with eras

shows the  of a custom CUDA implementation

blazingly fast, memory efficient vs. current libraries,
which are ancient fast C libraries with multiple bindings
and internal memory representation



Forum 200 https://forum.numer.ai/t/200-trees-per-second-with-cuda/7823
200 Code https://colab.research.google.com/drive/1EU0a4mMLyUBCBLdjIS3_qEM85bHERtKi
Forum TC: https://forum.numer.ai/t/a-closed-form-expression-for-tc/6280

warpGBM

https://github.com/jefferythewind/warpgbm

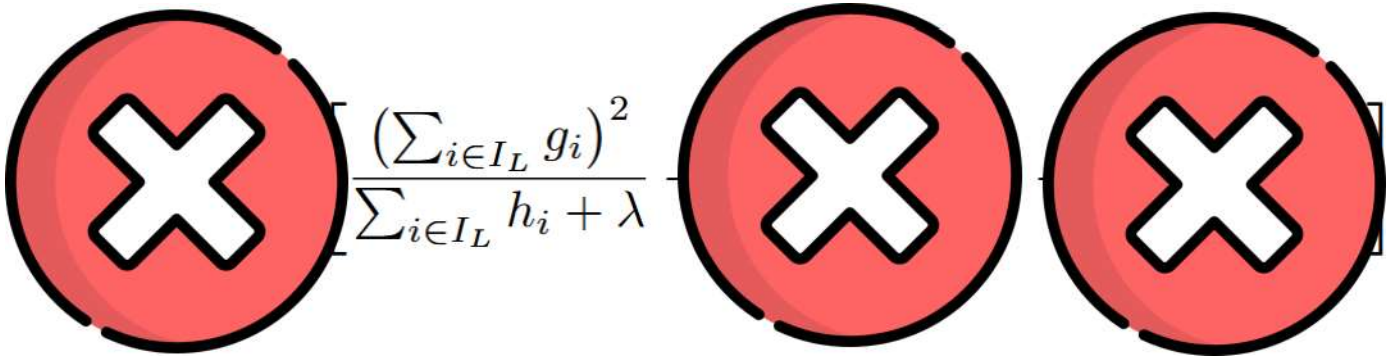
1. written in only python and cuda
2. impements directional era splitting
3. does not need to copy the input data

The Splitting Criterion

$$\mathcal{L}_{split} = \frac{1}{2}\left[\frac{\left(\sum_{i\in I_L} g_i\right)^2}{\sum_{i\in I_L} h_i + \lambda} + \frac{\left(\sum_{i\in I_R} g_i\right)^2}{\sum_{i\in I_R} h_i + \lambda} - \frac{\left(\sum_{i\in I} g_i\right)^2}{\sum_{i\in I} h_i + \lambda}\right]$$

Tianqi Chen and Carlos Guestrin. XGBoost : A Scalable Tree Boosting System. https://dl.acm.org/doi/10.1145/2939672.2939785

The Splitting Criterion



Tianqi Chen and Carlos Guestrin. XGBoost : A Scalable Tree Boosting System. https://dl.acm.org/doi/10.1145/2939672.2939785

The Splitting Criterion

$$\frac{\text{the sum of the gradients of the data}}{\text{the sum of the 2nd order gradients of the data}}$$

Tianqi Chen and Carlos Guestrin. XGBoost : A Scalable Tree Boosting System. https://dl.acm.org/doi/10.1145/2939672.2939785

The Splitting Criterion

the sum of the gradients of the data

---

the sum of the 2nd order gradients of the data

square to cancel signs

$$\frac{\left(\sum_{i\in I_L} g_i\right)^2}{\sum_{i\in I_L} h_i + \lambda}$$

regularization penalty

$$\mathcal{L}_{split} = \frac{1}{2}\left[\frac{\left(\sum_{i\in I_L} g_i\right)^2}{\sum_{i\in I_L} h_i + \lambda} + \frac{\left(\sum_{i\in I_R} g_i\right)^2}{\sum_{i\in I_R} h_i + \lambda} - \frac{\left(\sum_{i\in I} g_i\right)^2}{\sum_{i\in I} h_i + \lambda}\right]$$

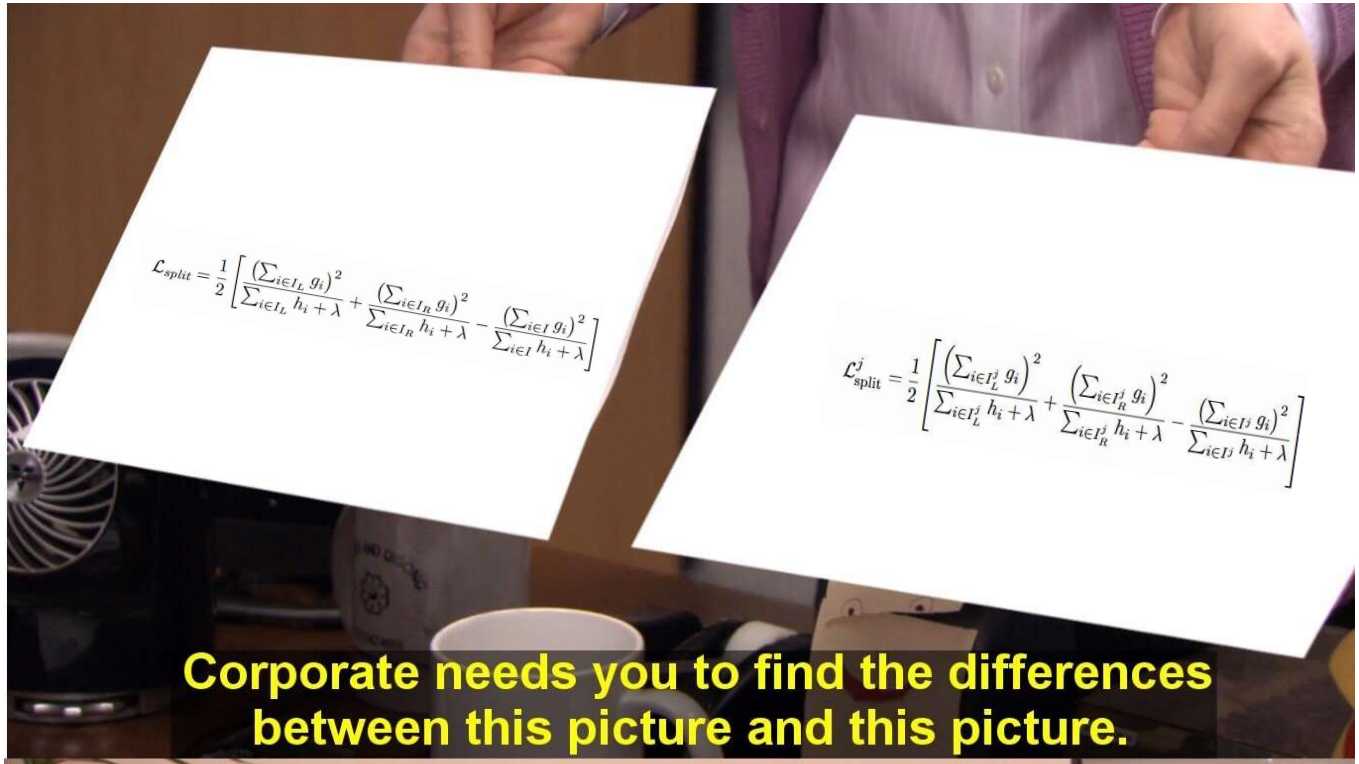
left node + right node - all data = favor even splits

Tianqi Chen and Carlos Guestrin. XGBoost : A Scalable Tree Boosting System. https://dl.acm.org/doi/10.1145/2939672.2939785

The Era Splitting Criterion

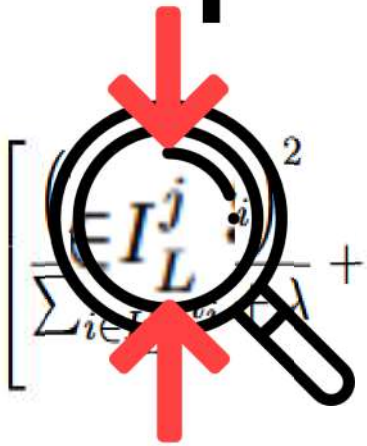
$$\mathcal{L}^j_{\text{split}} = \frac{1}{2}\left[\frac{\left(\sum_{i\in I^j_L} g_i\right)^2}{\sum_{i\in I^j_L} h_i + \lambda} + \frac{\left(\sum_{i\in I^j_R} g_i\right)^2}{\sum_{i\in I^j_R} h_i + \lambda} - \frac{\left(\sum_{i\in I^j} g_i\right)^2}{\sum_{i\in I^j} h_i + \lambda}\right]$$

https://arxiv.org/abs/2309.14496


$\mathcal{L}_{split} = \frac{1}{2}\left[\frac{(\sum_{i\in I_L} g_i)^2}{\sum_{i\in I_L} h_i + \lambda} + \frac{(\sum_{i\in I_R} g_i)^2}{\sum_{i\in I_R} h_i + \lambda} - \frac{(\sum_{i\in I} g_i)^2}{\sum_{i\in I} h_i + \lambda}\right]$
$\mathcal{L}^j_{split} = \frac{1}{2}\left[\frac{\left(\sum_{i\in I^j_L} g_i\right)^2}{\sum_{i\in I^j_L} h_i + \lambda} + \frac{\left(\sum_{i\in I^j_R} g_i\right)^2}{\sum_{i\in I^j_R} h_i + \lambda} - \frac{(\sum_{i\in I^j} g_i)^2}{\sum_{i\in I^j} h_i + \lambda}\right]$
Corporate needs you to find the differences between this picture and this picture.


They're the same picture.

The Era Splitting Criterion

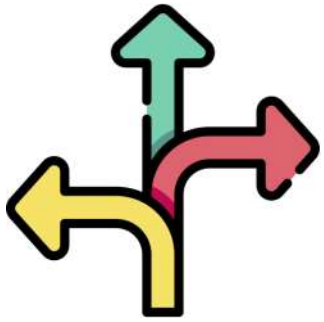


$$\mathcal{L}^j_{\text{split}} = \frac{1}{2}\left[\frac{\left(\sum_{i\in I^j_L} g_i\right)^2}{\sum_{i\in I^j_L} h_i + \lambda} + \frac{\left(\sum_{i\in I^j_R} g_i\right)^2}{\sum_{i\in I^j_R} h_i + \lambda} - \frac{\left(\sum_{i\in I^j} g_i\right)^2}{\sum_{i\in I^j} h_i + \lambda}\right]$$

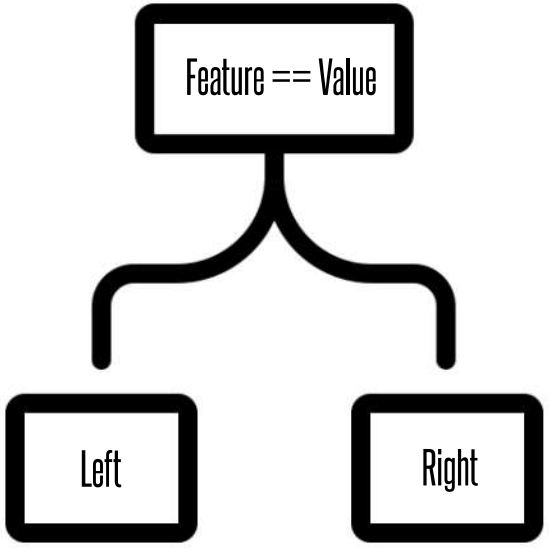
$$\mathcal{L}_{\text{era split}} = \frac{1}{M}\sum_{j=1}^{M} \mathcal{L}^j_{\text{split}}$$

The Era Splitting Criterion is the same as the original one, just done for each era seperately and then averaged together! The chosen split has the greatest average decrease of impurity over all eras.

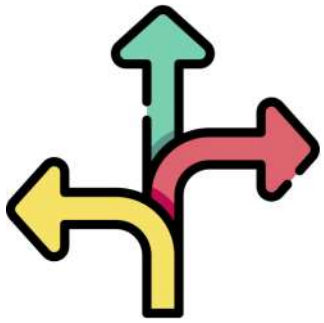
https://arxiv.org/abs/2309.14496



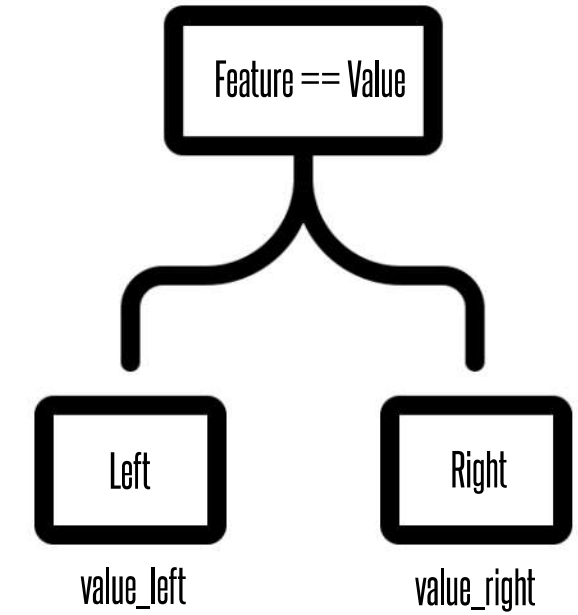
The DIRECTION of a Split



A split divides the data into a smaller (left) and bigger (right) portion. The prediction value of any node is the mean of all contained samples.



The DIRECTION of a Split

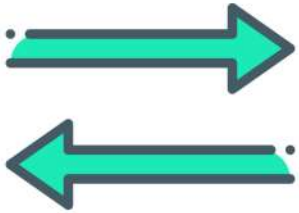


direction = "left" if value_left > value_right else "right"

A split divides the data into a smaller (left) and bigger (right) portion. The prediction value of any node is the mean of all contained samples.

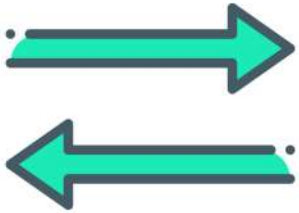
So every split has a DIRECTION:
LEFT, if the prediction of the left child is bigger
RIGHT, if the prediction of the right child is bigger

One Dataset, no problem. But the direction can be different for each ERA.



Directional Era Splitting

```
def directional_agreement(split):
    aggregated_direction = 0
    for era in eras:
        aggregated_direction += direction(split, era)
    return aggregated_direction / len(eras)
```

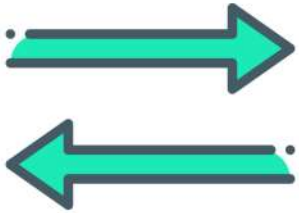


Directional Era Splitting

```
def directional_agreement(split):
    aggregated_direction = 0
    for era in eras:
        aggregated_direction += direction(split, era)
    return aggregated_direction / len(eras)
```

All eras split in the same direction: `directional_agreement(split) = 1`

Half eras split in the same direction: `directional_agreement(split) = 0`



Directional Era Splitting

```
def directional_agreement(split):
    aggregated_direction = 0
    for era in eras:
        aggregated_direction += direction(split, era)
    return aggregated_direction / len(eras)
```

All eras split in the same direction: `directional_agreement(split) = 1`

Half eras split in the same direction: `directional_agreement(split) = 0`

warpGBM Implementation

```
def choose_best_split(splits):
    max_agreement = max([directional_agreement(s) for s in splits])
    splits = [s for s in splits if directional_agreement(s) == max_agreement]
    return sorted(splits, key=split_criterion(s), reverse=True)
```

ERA vs Bucket

```
model = WarpGBM(...)
# no era splitting
era_id = np.ones(len(y))
# eras == buckets
era_id = numerai_data.erano
assert len(era_id) == len(y)
assert era_id.all(istype(int))
model.fit(X,y,era_id)
```

1 bucket for all data and 1 bucket for one era are the two extremes

Is it good?


CAUTION

What does it do?

The Experiment

train 922 eras [158,1079], test 80 eras [1096,1175]: 1 tree, learning_rate 1, colsample_bytree 1, depth 12

```
era_id = np.ones(len(y))
```

```
era_id = 10_Buckets
```

The Experiment

train 922 eras [158,1079], test 80 eras [1096,1175]: 1 tree, learning_rate 1, colsample_bytree 1, depth 12

```
era_id = np.ones(len(y))
```

3995 splits in 1 trees
1865/2376 features used
average directional agreement: 1

```
era_id = 10_Buckets
```

2098 splits in 1 trees
1335/2376 features used
average directional agreement: 1

The Experiment

train 922 eras [158,1079], test 80 eras [1096,1175]: 1 tree, learning_rate 1, colsample_bytree 1, depth 12

```
era_id = np.ones(len(y))
```

3995 splits in 1 trees
1865/2376 features used
average directional agreement: 1

KPI	Corr	MMC	Payout
Mean	0.006	0.004	0.012
Std	0.015	0.013	0.034
Sharpe	0.439	0.338	0.363
Sortino	-0.516	-0.520	-0.520
Drawdown	-0.029	-0.027	-0.069
Compound	1.561	1.346	1.905

```
era_id = 10_Buckets
```

2098 splits in 1 trees
1335/2376 features used
average directional agreement: 1

KPI	Corr	MMC	Payout
Mean	0.004	0.001	0.004
Std	0.013	0.011	0.028
Sharpe	0.330	0.117	0.169
Sortino	-0.499	-0.572	-0.538
Drawdown	-0.035	-0.030	-0.078
Compound	1.387	1.105	1.413

The Experiment II

train 922 eras [158,1079], test 80 eras [1096,1175]: 5 trees, learning_rate 0.2, colsample_bytree 1, depth 12

```
era_id = np.ones(len(y))
```

```
era_id = 10_Buckets
```

The Experiment II

train 922 eras [158,1079], test 80 eras [1096,1175]: 5 trees, learning_rate 0.2, colsample_bytree 1, depth 12

`era_id = np.ones(len(y))`

19859 splits in 5 trees
2367/2376 features used
average directional agreement: 1

`era_id = 10_Buckets`

9922 splits in 5 trees
2108/2376 features used
average directional agreement: 1

The Experiment II

train 922 eras [158,1079], test 80 eras [1096,1175]: 5 trees, learning_rate 0.2, colsample_bytree 1, depth 12

`era_id = np.ones(len(y))`

19859 splits in 5 trees
2367/2376 features used
average directional agreement: 1

KPI	Corr	MMC	Payout
Mean	0.007	0.003	0.010
Std	0.013	0.011	0.029
Sharpe	0.556	0.295	0.360
Sortino	-0.500	-0.564	-0.556
Drawdown	-0.024	-0.023	-0.059
Compound	1.715	1.277	1.936

`era_id = 10_Buckets`

9922 splits in 5 trees
2108/2376 features used
average directional agreement: 1

KPI	Corr	MMC	Payout
Mean	0.009	0.004	0.013
Std	0.014	0.011	0.030
Sharpe	0.680	0.367	0.445
Sortino	-0.424	-0.569	-0.538
Drawdown	-0.022	-0.020	-0.052
Compound	1.951	1.385	2.425

#buckets ?= #eras

example model: colsample = 0.1

directional agreement